Report

Track: Artificial Intelligence	**Assignment 2**
Name: Kamilya Shakirova	**Introduction to Big Data**
Email: k.shakirova@innopolis.university	

I. Methodology

In this project I built a small search engine that indexes text documents and ranks them by relevance to a user query. The system has several parts: data preparation, indexing with MapReduce, storing the index in Cassandra, and searching using PySpark with two ranking formulas (TF-IDF and BM25).

1.1 Data preparation

The original data is a Parquet file `a.parquet` containing many documents. I used a PySpark script (`prepare_data.py`) to take a sample of 100 documents. For each document I saved the title and text into a `.txt` file inside the `data/` folder. Then I wrote a single file with three tab-separated columns: `doc_id`, `title`, `text` and saved it to HDFS at `/input/data/part-00000`. This file is later used to map document IDs back to titles when showing search results.

1.2 Indexing with MapReduce (Hadoop Streaming)

I wrote a mapper (`mapper1.py`) and a reducer (`reducer1.py`) to create an inverted index.

- mapper: reads each line (`doc_id\ttext`), extracts all words (letters and numbers only, lowercased), and outputs `word\tdoc_id` for every occurrence.
- reducer: groups by word, counts how many times that word appears in each document, and outputs `word\tdoc_id\ttf` (term frequency).

The MapReduce job is launched by `create_index.sh` using Hadoop Streaming. The output is stored in HDFS at `/indexer/index/`. Each line of the output is a triple: `term \t doc_id \t tf`.

1.3 Storing the index in Cassandra

After the MapReduce job finishes, `store_index.sh` runs `app.py`. This Python script:

- connects to the Cassandra container (`cassandra-server`).
- creates a keyspace `search_engine` with replication factor 1 (enough for a small test).
- creates a table `inverted_index` with primary key `(term, doc_id)`.

- reads all output files from HDFS (`hdfs dfs -cat /indexer/index/part-*`) and inserts each triple into Cassandra using a prepared statement for speed.

1.4 Query processing and ranking

The search script `search.sh` takes a query string (e.g. "snowman christmas") and submits `query.py` to the YARN cluster.
Inside `query.py`:

- I read the whole inverted index from Cassandra into a PySpark RDD: each record is `(term, doc_id, tf)`.
- I compute the total number of documents `N`.
- for TF-IDF:
  - for each document, I sum over query terms: `tf * idf` where `idf = log((N+1)/(df+1))`.
  - `df` is the number of documents containing that term (counted from the RDD).
- for BM25 (with parameters `k1=1.5`, `b=0.75`):
  - I compute the average document length `avgdl` (sum of all term frequencies in a document, divided by number of documents).
  - the score for a document and a query term is:
    `idf * (tf * (k1+1)) / (tf + k1*(1 - b + b*doc_len/avgdl))`
    where `idf = log((N - df + 0.5)/(df + 0.5))`.
  - scores from all query terms are summed.

After calculating scores for all documents, I take the top 10 documents by score, join with the title file (`/input/data/part-00000`) to get the titles, and print the results.

1.5 Why two ranking functions?

I implemented both TF-IDF and BM25 to compare them. TF-IDF is simple and fast, but it does not take document length into account. BM25 is a more modern probabilistic model that normalises by document length and saturates term frequencies, which often gives better relevance. The output shows that BM25 reorders the top results slightly.

II. Demonstration

2.1 How to run the system

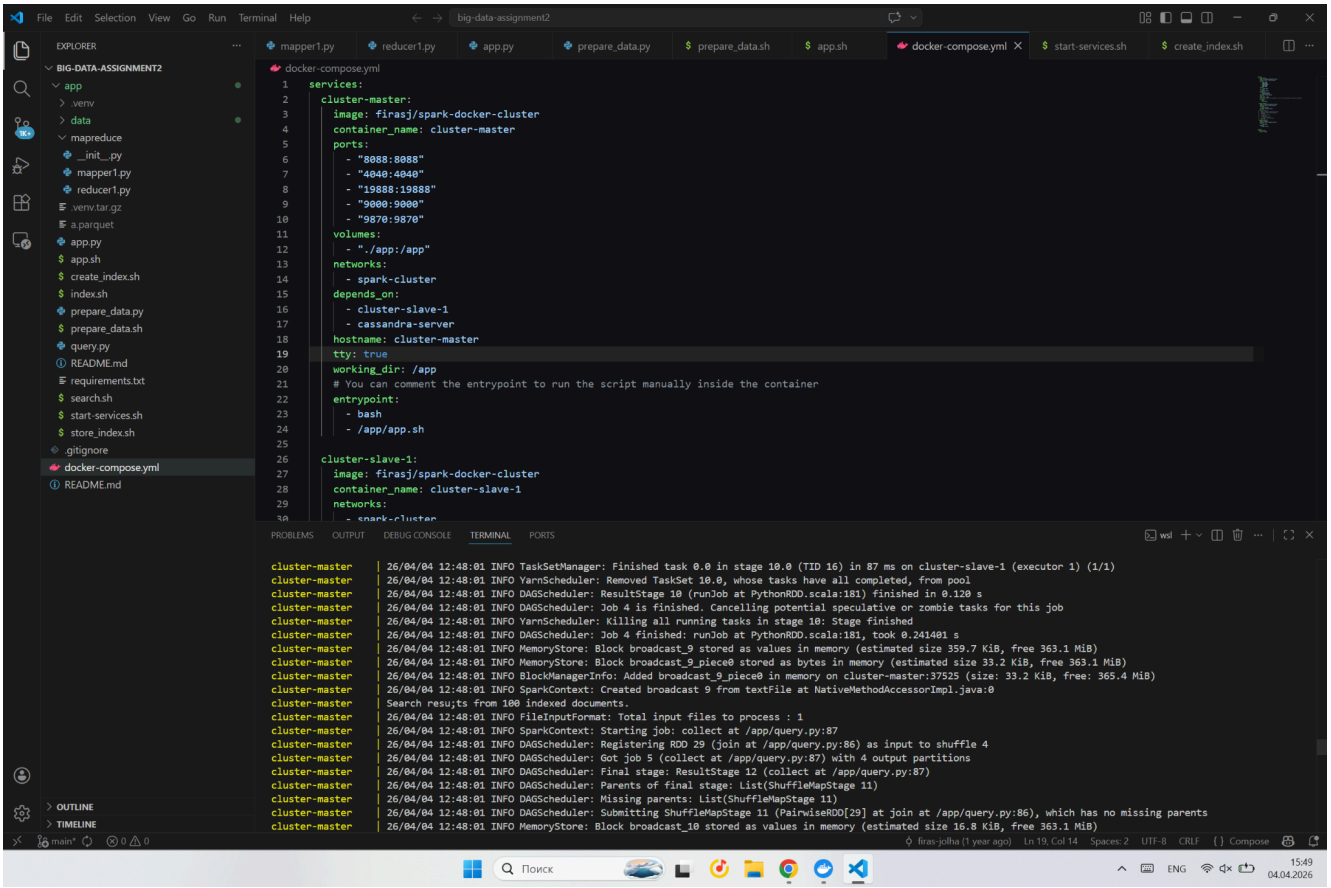
1. make sure Docker and Docker Compose are installed.
2. place the `a.parquet` file in the `app/` directory.
3. build and start the cluster:
    - docker compose up

The master container automatically runs `app.sh` as its entrypoint. The whole process is automatic. This script:

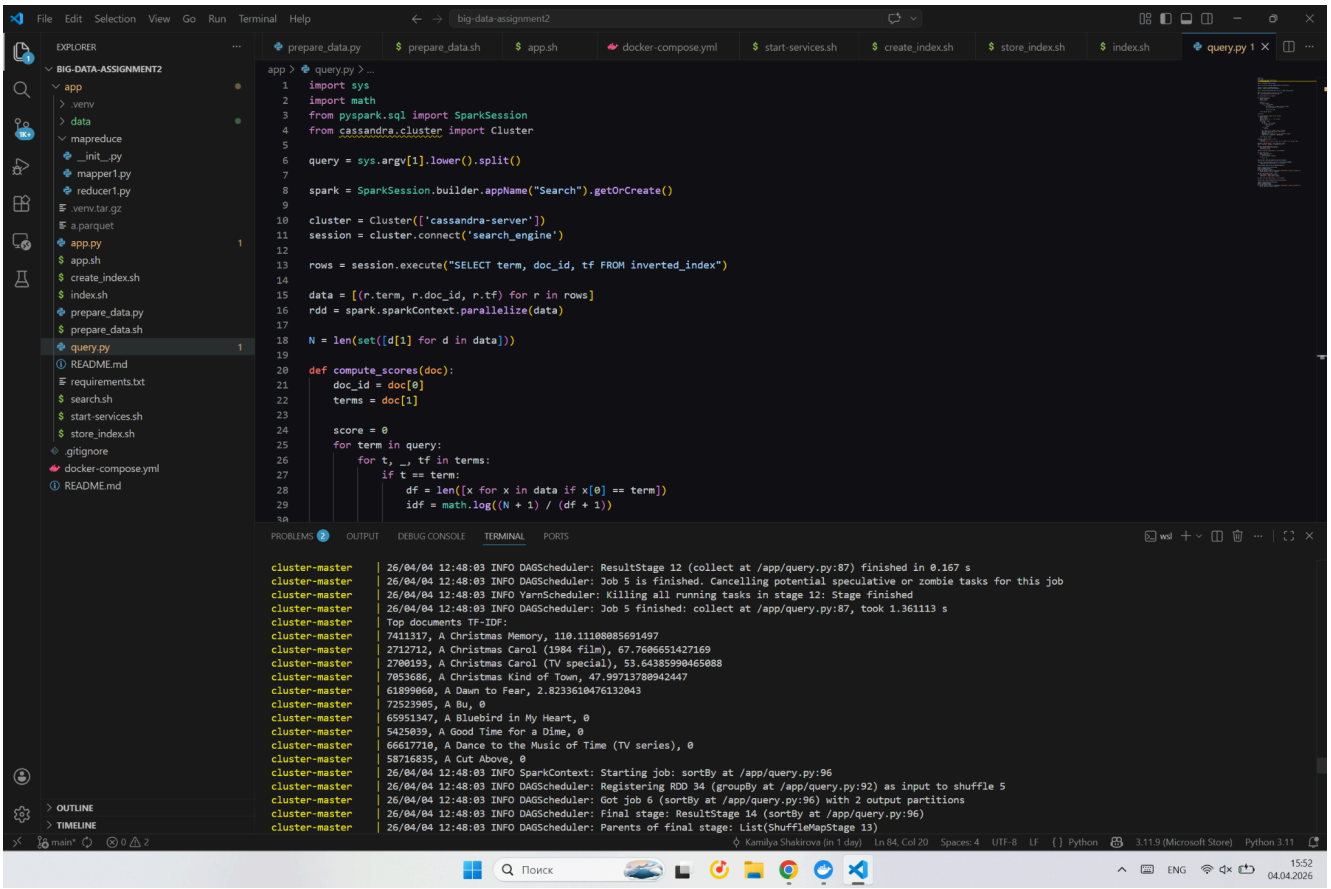
- starts Hadoop services (HDFS, YARN, HistoryServer).
- creates a Python virtual environment and installs dependencies (`cassandra-driver`, `pyspark`, etc.).
- packages the virtual environment with `venv-pack` (used later for YARN).
- runs `prepare_data.sh` → samples 100 documents and uploads them to HDFS.
- runs `index.sh` → launches MapReduce job and then stores the index into Cassandra.
- runs `search.sh "snowman christmas"` to query and show results.

2.2 Screenshots

Successful indexing of 100 documents

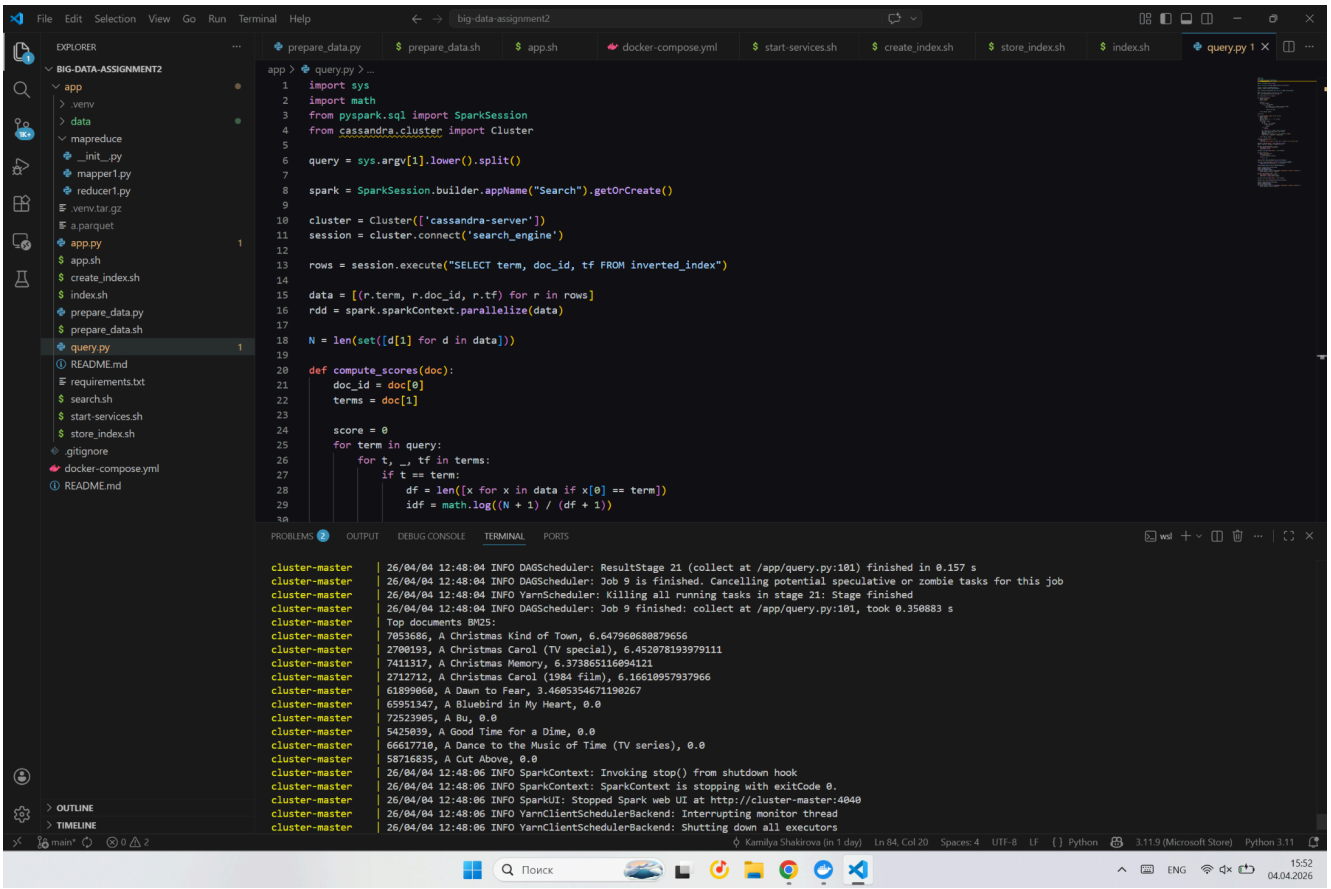


Search results for query "snowman christmas" (TF-IDF)



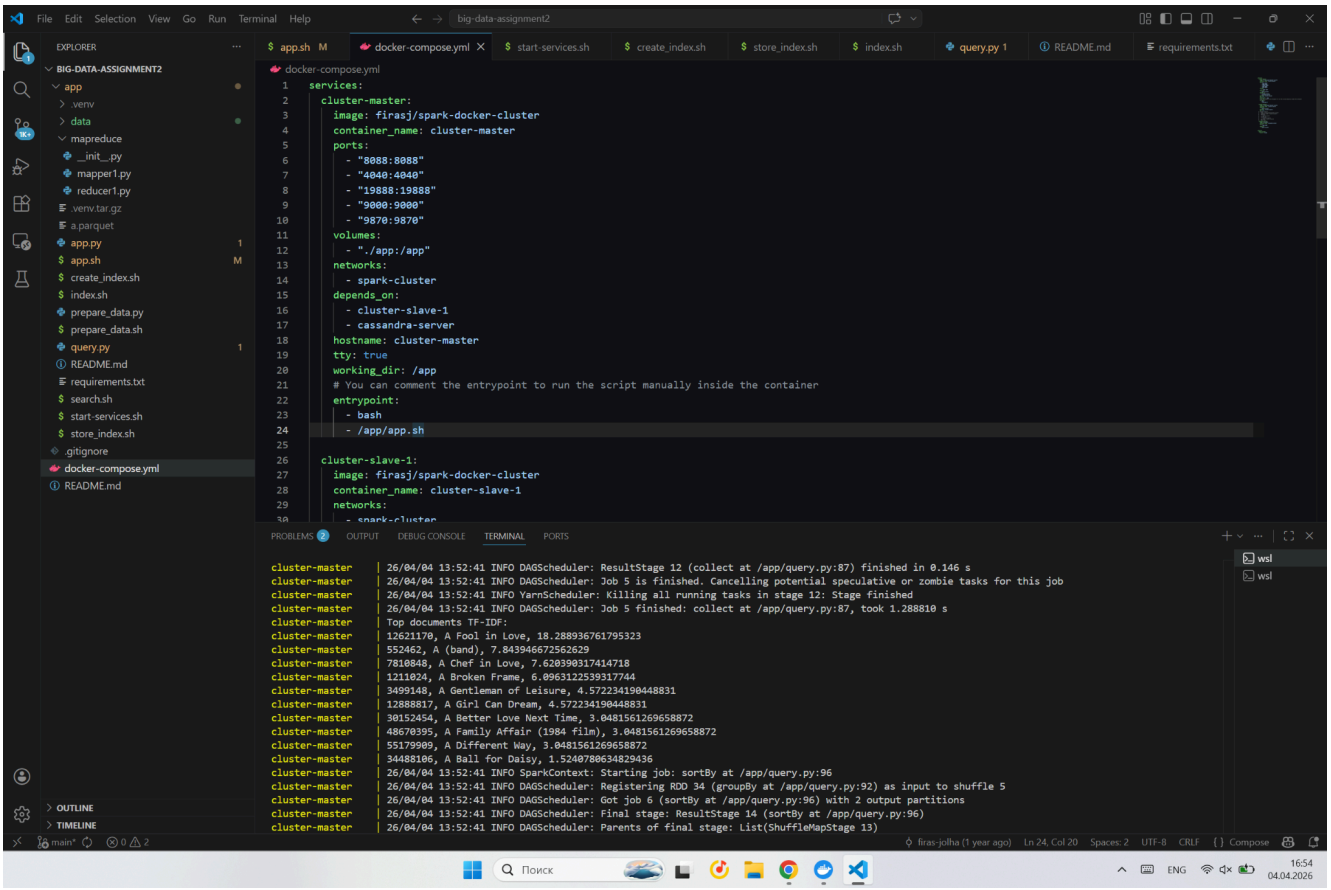
Top documents TF-IDF:
7411317, A Christmas Memory, 110.11108085691497
2712712, A Christmas Carol (1984 film), 67.7606651427169
2700193, A Christmas Carol (TV special), 53.64385990465088
7053686, A Christmas Kind of Town, 47.99713780942447
61899060, A Dawn to Fear, 2.8233610476132043
72523905, A Bu, 0
65951347, A Bluebird in My Heart, 0
5425039, A Good Time for a Dime, 0
66617710, A Dance to the Music of Time (TV series), 0
58716835, A Cut Above, 0

Search results for query "snowman christmas" (BM25)



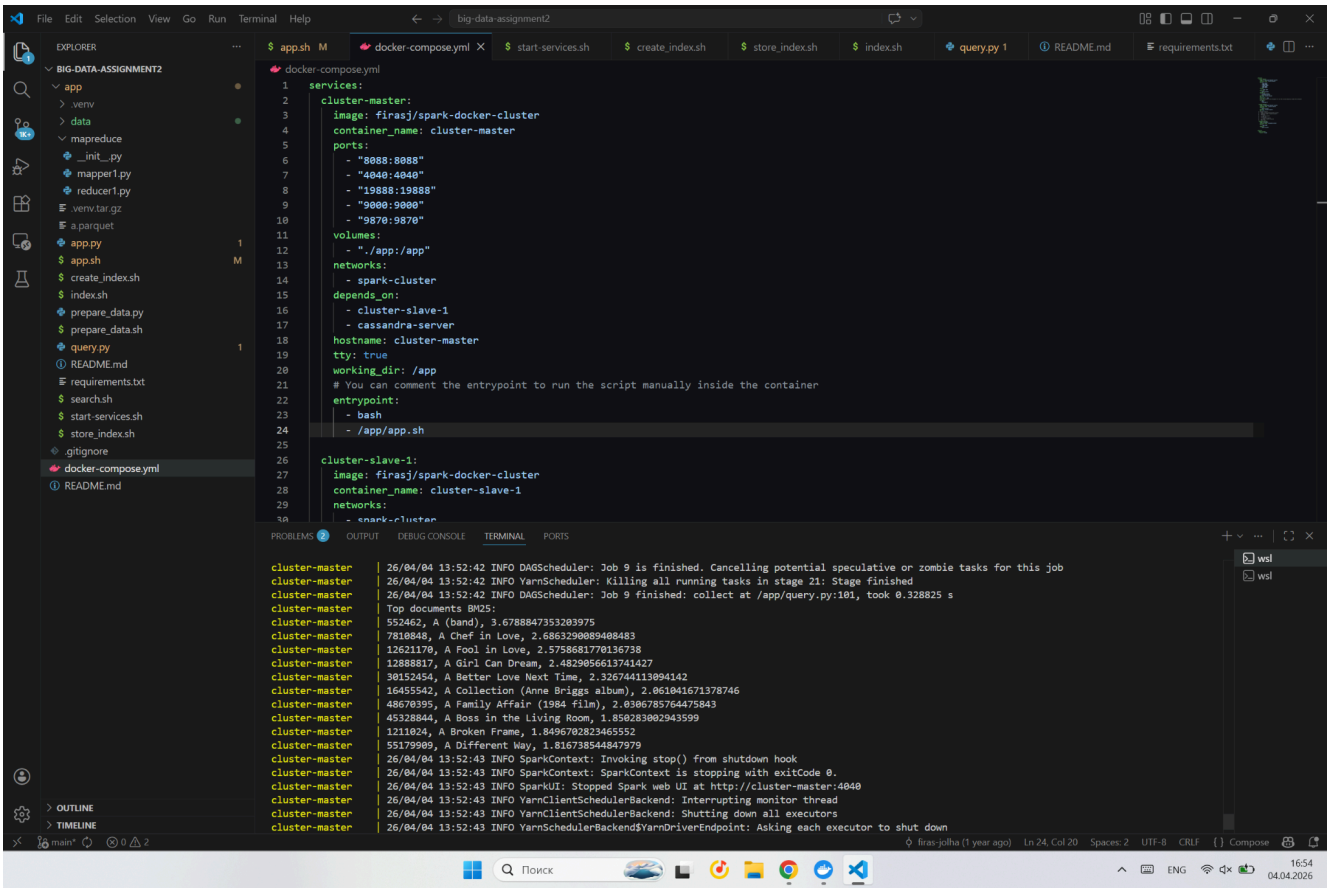
Top documents BM25:
7053686, A Christmas Kind of Town, 6.647960680879656
2700193, A Christmas Carol (TV special), 6.452078193979111
7411317, A Christmas Memory, 6.373865116094121
2712712, A Christmas Carol (1984 film), 6.16610957937966
61899060, A Dawn to Fear, 3.4605354671190267
65951347, A Bluebird in My Heart, 0.0
72523905, A Bu, 0.0
5425039, A Good Time for a Dime, 0.0
66617710, A Dance to the Music of Time (TV series), 0.0
58716835, A Cut Above, 0.0

Search results for query "warm love" (TF-IDF)



Top documents TF-IDF:
12621170, A Fool in Love, 18.288936761795323
552462, A (band), 7.843946672562629
7810848, A Chef in Love, 7.620390317414718
1211024, A Broken Frame, 6.0963122539317744
3499148, A Gentleman of Leisure, 4.572234190448831
12888817, A Girl Can Dream, 4.572234190448831
30152454, A Better Love Next Time, 3.0481561269658872
48670395, A Family Affair (1984 film), 3.0481561269658872
55179909, A Different Way, 3.0481561269658872
34488106, A Ball for Daisy, 1.5240780634829436

Search results for query "warm love" (BM25)



Top documents BM25:
552462, A (band), 3.6788847353203975
7810848, A Chef in Love, 2.6863290089408483
12621170, A Fool in Love, 2.5758681770136738
12888817, A Girl Can Dream, 2.4829056613741427
30152454, A Better Love Next Time, 2.326744113094142
16455542, A Collection (Anne Briggs album), 2.061041671378746
48670395, A Family Affair (1984 film), 2.0306785764475843
45328844, A Boss in the Living Room, 1.850283002943599
1211024, A Broken Frame, 1.8496702823465552
55179909, A Different Way, 1.816738544847979

2.3 Explanation of results

Query: "snowman christmas".

None of the documents in our sample contain the word "snowman" (I checked the titles and a few texts). Therefore all scores come only from the word "christmas".

- Documents that do not contain "christmas" get a score of 0 (the last five entries in both lists).
- Documents containing "christmas" are ranked.

Query: "warm love".

- `A (band)` has 2×"warm", 0×"love", length 7575.
- `A Chef in Love` has 0×"warm", 5×"love", length 1725.
- `A Fool in Love` has 0×"warm", 11×"love", length 6323.

TF-IDF (no length normalisation) puts `A Fool in Love` first because it has the most `love` occurrences.
BM25 (normalises by length) puts `A (band)` first because `warm` is rarer (higher IDF) and the length penalty is offset by the rare term; `A Chef in Love` wins `A Fool in Love` because it is shorter.